



Tecnológico de Monterrey

Campus Santa Fe

MA. Actividad: Roomba

Modelación de sistemas multiagentes con gráficas computacionales (Gpo 301)

Katia Abigail Álvarez Contreras - A01781097

22 - noviembre / 2025

Problema a resolver:

La situación se basa en diseñar y evaluar un sistema de agentes autónomos capaces de limpiar una habitación representada como una cuadrícula de $M \times N$ espacios, donde existen celdas sucias y obstáculos distribuidos aleatoriamente.

Cada agente debe actuar de manera eficiente bajo restricciones energéticas, ya que cuenta con una batería limitada que disminuye con cada acción y requiere regresar periódicamente a una estación de carga para continuar operando. El desafío implica crear agentes que, mediante una arquitectura basada en subsunción y capacidades adecuadas de percepción, proactividad y toma de decisiones, logren maximizar el porcentaje de limpieza dentro de un tiempo máximo de ejecución, evitando quedarse sin energía.

Además, se busca analizar cómo varía el rendimiento del sistema al comparar un solo agente con múltiples agentes, considerando métricas como tiempo de limpieza, movimientos realizados y cobertura alcanzada, con el fin de comprender el impacto de la cooperación o interferencia entre agentes en un entorno dinámico y parcialmente desconocido.

Propuesta:

La propuesta de solución consiste en diseñar agentes autónomos tipo Roomba capaces de limpiar un entorno cuadrícula mientras gestionan de forma eficiente su energía y cooperan cuando es necesario.

Cada agente opera mediante una arquitectura de subsunción que prioriza comportamientos esenciales como evitar quedarse sin batería, regresar a una estación de carga y evitar obstáculos, sobre actividades de mayor nivel como la exploración, limpieza y comunicación. El sistema combina percepción local del entorno, toma de decisiones reactiva y estrategias proactivas para seleccionar celdas óptimas según su nivel de conocimiento y suciedad. Además, cuando existen múltiples agentes, estos comparten información sobre celdas visitadas y estaciones de carga descubiertas, mejorando la cobertura global. Con estas

capacidades, el modelo busca maximizar la cantidad de celdas limpias dentro del tiempo disponible, evitando que los agentes queden inactivos por batería agotada y manteniendo un comportamiento adaptable ante incertidumbre y obstáculos.

Agentes:

- Objetivo: Limpiar la mayor cantidad posible de celdas dentro del tiempo máximo de ejecución, evitando que el agente se quede sin batería mediante ciclos eficientes de limpieza, exploración y recarga.
- Capacidad Efectora:
 - Movimiento a celdas vecinas no bloqueadas.
 - Limpieza de celdas sucias.
 - Retorno a estaciones de carga usando BFS.
 - Carga de batería.
 - Comunicación y transferencia de información con agentes vecinos.
- Percepción:
 - Detección del estado de la celda actual (Dirty, Clean, Obstacle, Charger).
 - Identificación de obstáculos en celdas adyacentes.
 - Reconocimiento de otros RoombaAgents en la misma celda para comunicación.
 - Registro de celdas visitadas y estaciones de carga conocidas.
- Proactividad:
 - Exploración orientada, priorizando celdas desconocidas y sucias para maximizar efectividad.
 - Selección estratégica de movimientos mediante la clasificación unknown/dirty, unknown, known/dirty, known.

- Decisión anticipada de regresar al cargador cuando la batería cae por debajo del umbral de seguridad (20%).
- Reactividad:
 - Interrupción inmediata de limpieza y activación del modo returning cuando la batería es baja.
 - Cambio a charging al detectar una estación de carga.
 - Evitación instantánea de obstáculos.
 - Cambio de estado según condiciones del entorno o del agente (suciedad, comunicación, batería).
- Habilidad social:
 - Intercambio bidireccional de celdas visitadas.
 - Compartición de estaciones de carga detectadas.
 - Coordinación emergente mediante información compartida, reduciendo exploración redundante.
- Métricas de desempeño:
 - Porcentaje de celdas limpias al final de la simulación.
 - Tiempo (en pasos) requerido para limpiar el entorno o llegar al límite temporal.
 - Número de movimientos realizados por agente.
 - Batería promedio del sistema.
 - Cantidad total de celdas sucias, limpias, obstáculos y estaciones de carga.
- Acciones:
 - `clean()`: limpia la celda actual si está sucia.
 - `move()`: avanza hacia una celda no obstáculo, priorizando según conocimiento.
 - `return_to_charger()`: inicia retorno a estación usando BFS.

- `charge()`: recupera energía si está sobre un cargador.
- `communicate()`: comparte información con agentes en la misma celda.
- `move_way()`: sigue rutas calculadas mediante BFS.
- Estados:
 - `cleaning`: limpiando celdas o explorando.
 - `returning`: volviendo a una estación de carga.
 - `charging`: recargando batería.
 - `communicating`: compartiendo información con otros agentes.
 - `idle`: sin acciones relevantes.
 - `dead`: sin batería para operar.
- Arquitectura de subsunción (organizada en niveles jerárquicos):
 1. Supervivencia (prioridad máxima)
 - Gestionar batería baja → activar *returning*.
 - Detectar cargador → activar *charging*.
 - Evitar obstáculos.
 2. Limpieza y exploración
 - Si la celda está sucia → limpiar.
 - Si no está sucia → explorar utilizando el modelo de prioridad.
 3. Comunicación (nivel superior pero de baja prioridad)
 - Compartir estaciones de carga y celdas visitadas cuando coincide con otros agentes.
 - Volver automáticamente al estado *cleaning* tras comunicar.

Esta jerarquía garantiza que las necesidades vitales del agente dominan sobre comportamientos complejos, asegurando operación continua y robusta.

Ambiente:

- Accesible vs. Inaccesible: El ambiente es Inaccesible, ya que el agente no puede obtener información completa del entorno; únicamente percibe el estado de la celda en la que se encuentra y de sus vecinos inmediatos. No conoce la distribución global de la suciedad, los obstáculos ni la localización de todas las estaciones de carga, a menos que las descubra o las reciba por comunicación.
- Determinista vs. No determinista: El ambiente es no determinista porque aunque las acciones del agente tienen un efecto esperado, su resultado no está 100% garantizado. El movimiento puede verse limitado por obstáculos inesperados o rutas no disponibles, lo que provoca que la acción no siempre conduzca a la celda deseada. Además, en un sistema multiagente, la presencia de otros agentes introduce variabilidad en los resultados.
- Estático vs. Dinámico: El ambiente es estático debido a que no cambia por sí mismo; únicamente evoluciona a partir de las acciones de los agentes. La suciedad no aparece de nuevo, los obstáculos no cambian de lugar y las estaciones de carga permanecen constantes.
- Discreto vs. Continuo: El entorno está formado por una cuadrícula dividida en celdas discretas, y tanto las acciones como las percepciones del agente son finitas: moverse a un vecino, limpiar, regresar al cargador, comunicar, cargar, etc. No existe variación continua de posiciones o sensores.

PEAS del ambiente y los agentes:

- P – Performance (Medida de desempeño):
 - Los agentes buscan maximizar la limpieza del entorno mientras minimizan el riesgo de quedarse sin batería. Sus criterios de desempeño incluyen limpiar la

mayor cantidad de celdas posibles, evitar obstáculos, regresar a tiempo a una estación de carga, mantener un número razonable de movimientos y coordinarse eficientemente con otros agentes cuando es necesario.

- E – Environment (Ambiente):

- El entorno de los agentes es una cuadrícula bidimensional compuesta por celdas que pueden estar en estado sucio, limpio, obstáculo o actuar como estación de carga. Tanto la suciedad como los obstáculos se generan de manera aleatoria, lo que introduce incertidumbre y hace que cada simulación sea diferente. En el caso multiagente, hay múltiples estaciones de carga distribuidas aleatoriamente.

- A – Actuadores:

- Los actuadores permiten a cada agente interactuar físicamente con su entorno.

En este modelo, los actuadores incluyen:

- Movimiento hacia cualquier celda vecina no bloqueada.
- Limpieza de la celda actual.
- Retorno guiado mediante rutas calculadas con BFS hacia estaciones de carga.
- Comunicación con otros agentes ubicados en la misma celda.
- Decisión de cargar cuando se encuentra en una estación.

- Estos actuadores permiten al agente cumplir su objetivo de limpieza manteniendo su supervivencia energética.

- S – Sensores:

- Los sensores proporcionan al agente la capacidad de percibir localmente su entorno. Incluyen:

- La “vista” de la celda actual y de las celdas vecinas inmediatas.

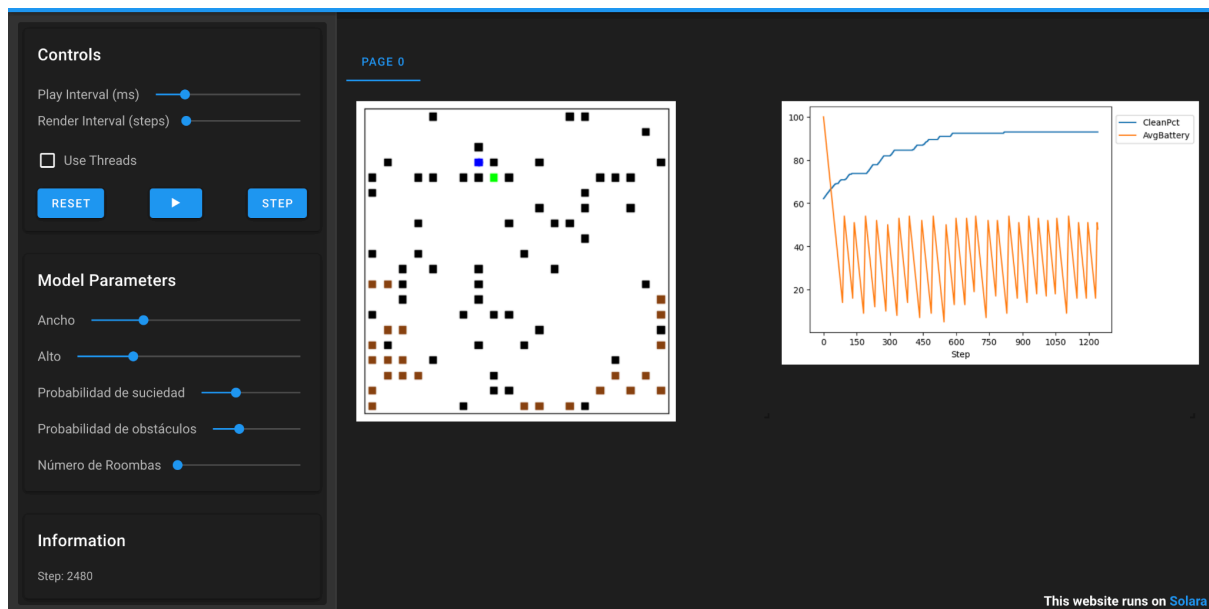
- Detección del estado del piso (Dirty, Clean, Obstacle, Charger).
 - Percepción de otros RoombaAgents en la misma celda para iniciar comunicación.
 - Reconocimiento del nivel de batería del propio agente.
- Gracias a estos sensores, el agente puede tomar decisiones informadas y adaptarse al ambiente incluso cuando este es parcialmente observable.

Estadísticas:

- Tiempo necesario hasta que todas las celdas estén limpias (o se haya llegado al tiempo máximo).
- Porcentaje de celdas limpias después del término de la simulación.
- Número de movimientos realizados por el agente.

Simulación 1:

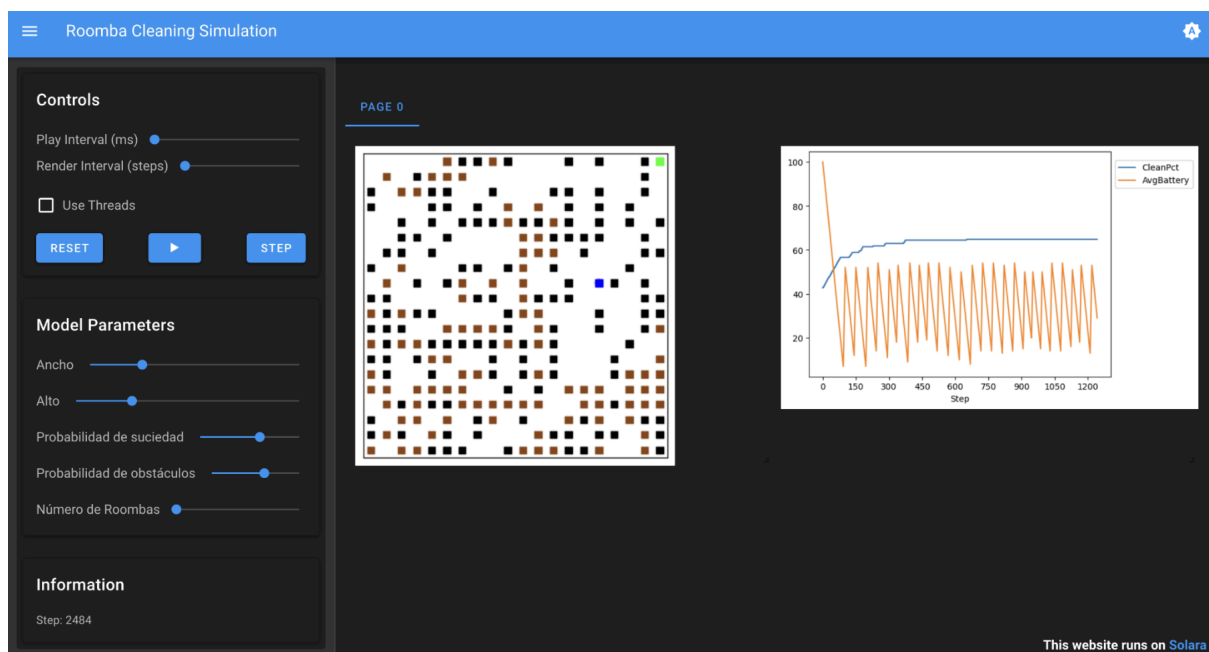
- **Probabilidad de suciedad del 35% y obstáculos del 15%**



En este primer caso, el Roomba logró limpiar gran parte del entorno durante los primeros cientos de pasos, alternando entre limpiar, explorar y recargar su batería. Sin

embargo, conforme avanzó la simulación, llegó a un punto en el que el porcentaje de limpieza dejó de aumentar, aun cuando el agente seguía moviéndose y cargándose continuamente. Esto se debe a que, la suciedad restante quedó ubicada en zonas difíciles de alcanzar o rodeadas por obstáculos, y el agente al depender únicamente de percepción local y exploración basada en conocimiento parcial no pudo encontrarla. Como resultado, la eficiencia de limpieza se estancó alrededor del paso 2480, mostrando la limitación natural de un agente individual en entornos complejos.

- **Probabilidad de suciedad del 60% y obstáculos del 30%**

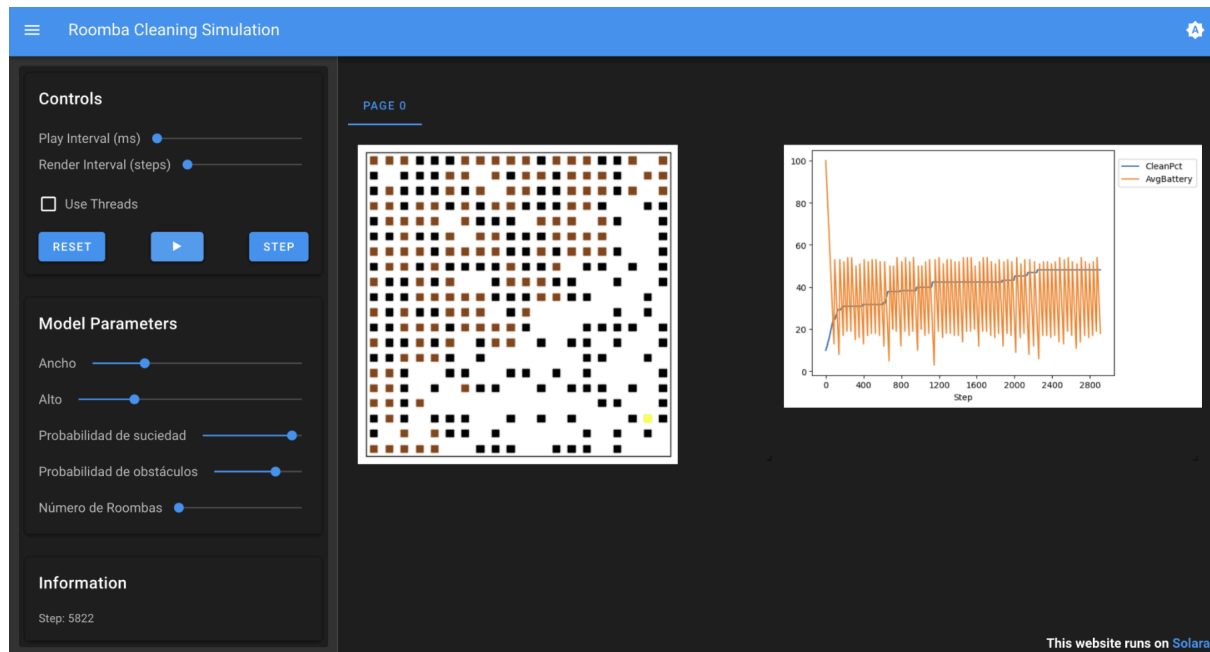


El agente individual enfrenta un entorno mucho más denso y complejo, lo que limita significativamente su capacidad de exploración. En las primeras etapas de la simulación, el porcentaje de limpieza aumenta rápidamente, pero pronto se estabiliza cerca del 65%, ya que la alta concentración de obstáculos bloquea rutas y deja zonas inaccesibles o difíciles de descubrir mediante percepción local.

Aunque el agente continúa moviéndose, limpiando y recargando su batería de forma cíclica, como se observa en las oscilaciones de la gráfica, ya no logra acceder a la suciedad restante. Esto evidencia que, en entornos altamente obstruidos, un agente único rápidamente

alcanza su límite operativo y no puede completar la limpieza debido a la falta de rutas viables y a su conocimiento parcial del entorno.

- **Probabilidad de suciedad del 90% y obstáculos del 35%**



El agente individual enfrenta un entorno extremadamente saturado, donde la movilidad está fuertemente restringida y la mayoría de las rutas están bloqueadas. En los primeros pasos el porcentaje de limpieza aumenta lentamente, pero pronto queda limitado alrededor del 45–50%, ya que gran parte de la suciedad se encuentra detrás de barreras o en zonas inaccesibles. La gráfica muestra que el agente continúa moviéndose y recargando la batería de forma repetitiva, pero su avance es prácticamente nulo debido a la falta de caminos libres y a su percepción local, que no le permite planificar rutas alternativas complejas. Como resultado, incluso después de más de 5800 pasos, la limpieza se estanca y el agente demuestra su incapacidad natural para completar la tarea en ambientes altamente obstruidos y con alta densidad de suciedad.

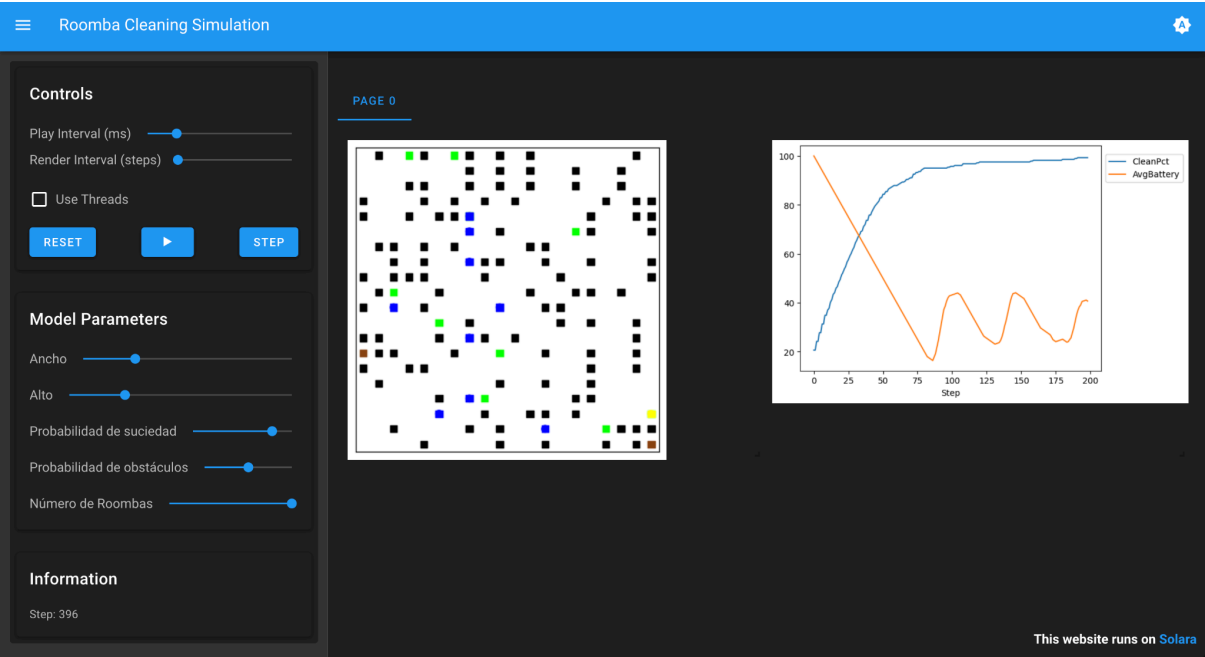
Simulación 2:

- **Todos con la misma probabilidad de suciedad y obstáculos**

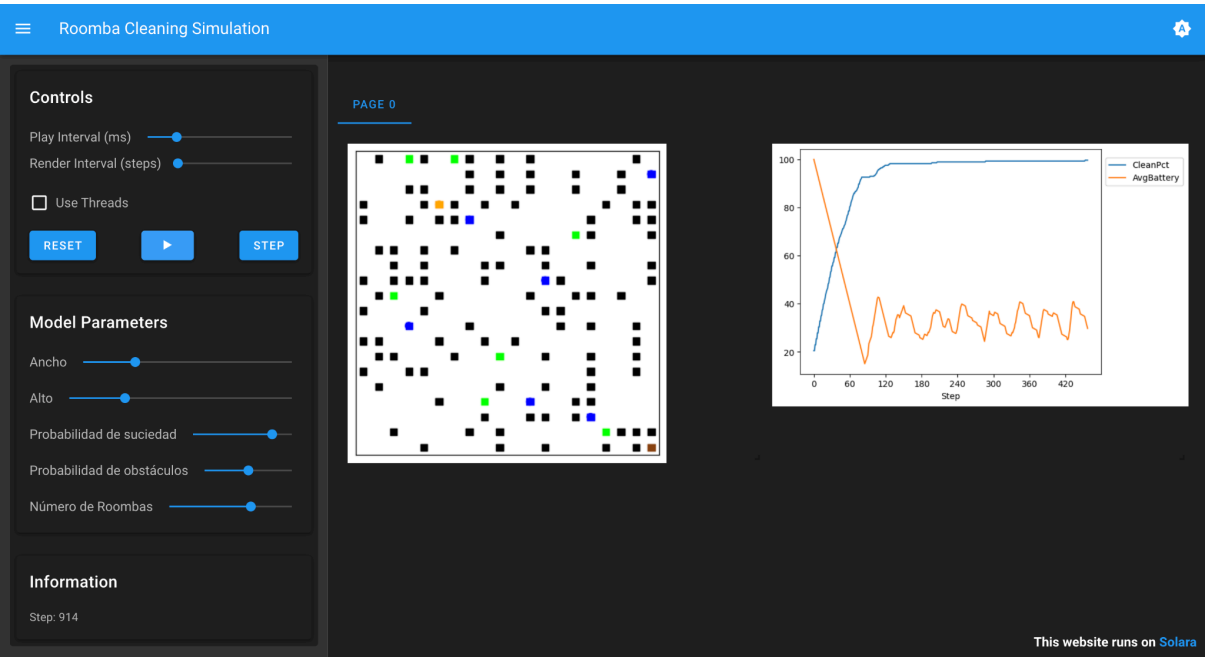
En las simulaciones donde se mantuvieron constantes las probabilidades de suciedad y obstáculos, el número de agentes fue el factor determinante en la velocidad de limpieza. Los resultados muestran que, al aumentar la cantidad de Roombas, el tiempo total para alcanzar altos niveles de limpieza disminuye considerablemente. Por ejemplo, con 3 agentes la simulación requiere alrededor de 1922 steps, mientras que con 5 agentes el proceso se reduce a 814 steps, y con 7 agentes baja incluso a cerca de 914 steps. La diferencia más notable ocurre con 10 agentes, que logran limpiar casi por completo en solo 396 steps, evidenciando una exploración paralela mucho más eficiente.

Sin embargo, el beneficio de aumentar el número de agentes no crece de forma totalmente lineal. Aunque más Roombas aceleran la limpieza, también generan mayor congestión, solapamiento de rutas y movimientos redundantes, lo que se refleja en variaciones más pronunciadas en la gráfica de batería promedio. Aun con estas interferencias, los tiempos finales muestran que incorporar múltiples agentes mejora drásticamente el desempeño frente a un solo agente, permitiendo cubrir el entorno más rápidamente y disminuir las zonas sin explorar, especialmente en mapas con obstáculos dispersos que dificultan la navegación individual.

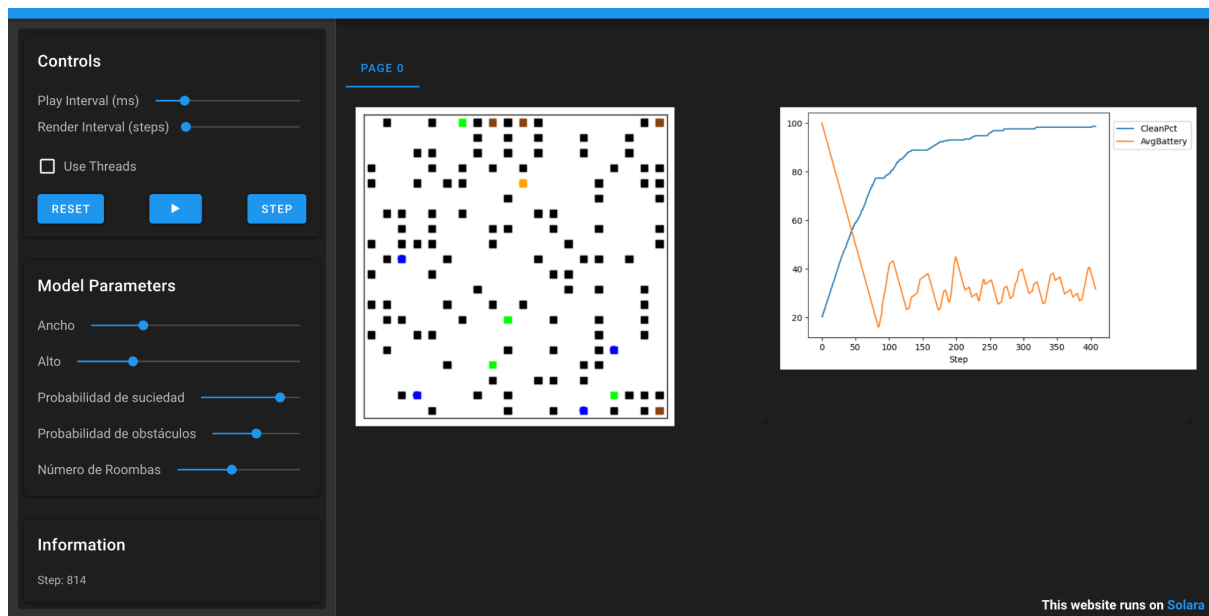
- 10 agentes



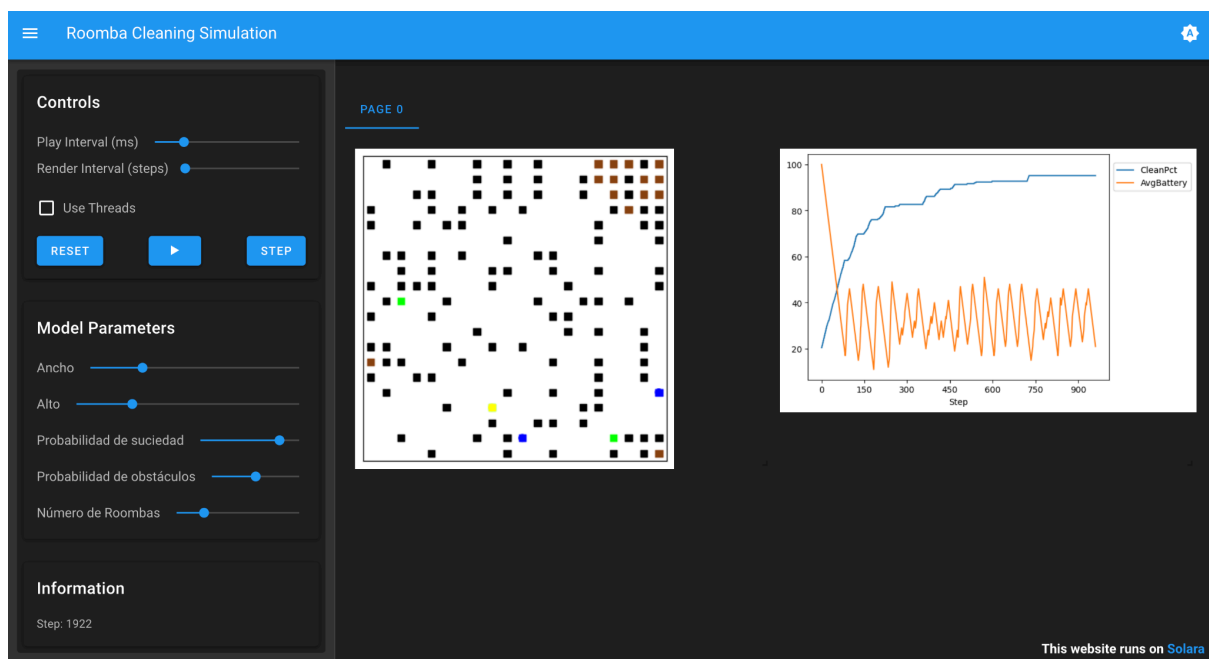
- 7 agentes



- 5 agentes



- 3 agentes



Conclusiones:

Las simulaciones realizadas permiten observar que el desempeño de los agentes Roomba está fuertemente condicionado por las características del entorno y por la cantidad de agentes disponibles. Un solo agente es capaz de limpiar eficazmente durante las primeras etapas, pero su percepción limitada y la presencia de obstáculos provocan que eventualmente se estanque sin poder localizar la suciedad restante, especialmente en ambientes densos o con

alto bloqueo. En cambio, cuando se incorporan múltiples agentes, la exploración del entorno se acelera significativamente y el tiempo total de limpieza se reduce de forma notable gracias al trabajo simultáneo y a la capacidad de compartir información sobre celdas visitadas y estaciones de carga.

Sin embargo, agregar más agentes también introduce nuevos retos, como congestión en áreas estrechas y movimientos redundantes, lo que limita parcialmente el beneficio de incrementar su número más allá de cierto punto.

En conjunto, las simulaciones demuestran que la cooperación, la percepción local y la arquitectura de subsunción permiten que los agentes alcancen un desempeño robusto, aunque siempre dentro de las limitaciones impuestas por el diseño del entorno y los recursos energéticos.